# check_cm_test_mock_pid_explicit_int

# scripts/pre_build/check_cm_test_mock_pid_explicit_int.sh — CM-TEST-MOCK-PID-EXPLICIT-INT static pre-build gate

**Revision:** 1 **Last modified:** 2026-08-19T00:00:00Z **Status:** active **Item:** BOB-126 follow-up (§11.4.238 discovery-channel-escape closure — a new automated check for a defect class first found out-of-band); sibling of `scripts/pre_build/check_cm_killpg_pgid_guard.sh` (CM-KILLPG-PGID-GUARD), which guards **production** code — this gate guards the **test** side of the same defect class.

## Purpose

`scripts/pre_build/check_cm_test_mock_pid_explicit_int.sh` statically detects **test files** that construct an `AsyncMock()` / `MagicMock()` standing in for an `asyncio.subprocess.Process` (identified by nearby `.stdout.readline` / `.stderr.read` / `.wait` / `.kill` usage, combined with an explicit `.returncode = None` — see "Detection design" below) but never explicitly set `.pid = <int>` on it, and never neutralise the destructive call by patching `os.killpg`.

### The defect class this closes (BOB-126)

An unconfigured `unittest.mock` object auto-generates every magic method, including `__int__` / `__index__`, both **defaulting to 1**. A test double standing in for a real subprocess whose `.pid` attribute is left unconfigured therefore coerces to the integer `1` wherever production code reads `int(proc.pid)` — and `os.killpg(os.getpgid(1), SIGKILL)` is, under glibc, **identical** to `kill(-1, SIGKILL)`: a broadcast kill of every process the test-runner's UID owns. This is the exact root cause traced across 7 forced logouts (BOB-116/120/123/124/125/126).

`download-proxy/src/merge_service/search.py` now carries a production-side guard (`isinstance(_pid, int) and _pid > 1`), mechanically enforced by the sibling gate `check_cm_killpg_pgid_guard.sh`. This gate makes the **test** half of the same hardening — never relying on a mocked `.pid` that could coerce to `1` — a mechanically enforced invariant instead of tribal knowledge, per §11.4.226 (evidence-class-at-closure) and §11.4.240/§11.4.249 (a producer that also authors its own oracle is unverified — this gate is the structurally-separate, independent check on the TEST code).

## Usage

```
# Default scope: tests/**/*.py (relative to the repo root this
        script
# resolves from its own location), excluding __pycache__.
scripts/pre_build/check_cm_test_mock_pid_explicit_int.sh

# Explicit-path mode — scan exactly the given file(s)/
        directory(ies)
# instead of the default scope. Used by the meta-test to run
        against
# hermetic fixtures.
scripts/pre_build/check_cm_test_mock_pid_explicit_int.sh /path/to/
        file.py
scripts/pre_build/check_cm_test_mock_pid_explicit_int.sh /path/to/
        fixture/dir

# Verbose (prints every skip decision, not just findings):
scripts/pre_build/check_cm_test_mock_pid_explicit_int.sh -v

scripts/pre_build/check_cm_test_mock_pid_explicit_int.sh --help
```

# Inputs

- **Positional arguments** (optional): zero or more file/directory paths.
  - Zero arguments → default scope: `tests/` under the repository root, walked recursively for `*.py` files.
  - One or more arguments → scan exactly those paths instead (single files must end in `.py` to be considered; directories are walked with the same `__pycache__` exclude as the default scope).
- **`-v`** / **`--verbose`**: also print each skip decision (no-returncode-None, no-lifecycle-indicator, already-guarded, killpg-exempt).
- **`-h`** / **`--help`**: print the in-source documentation header and exit 0.
- No environment variables are read.

# Outputs

- **stdout**: a run header (scope, window sizes, file count) and either `PASS: CM-TEST-MOCK-PID-EXPLICIT-INT clean across N file(s)` or (on `-v`) per-candidate skip lines.
- **stderr** (only on failure): a `=== FINDINGS ===` block, one `FAIL: <path>:<line>: unguarded subprocess-mock — <trimmed source> [identifier=<ident>] (no \`.pid = ` and no `os.killpg` patch within 30 lines)`line per unguarded hit, followed by a`FAIL: unguarded test-subprocess-mock pid hit(s) …` summary line.
- **Exit codes**:
  - `0` — zero unguarded hits (or, in explicit-path mode, zero matching files — an empty scope is not itself a finding).
  - `1` — one or more unguarded hits; see stderr for the per-hit report.
  - `2` — usage/environment error (unknown flag, or an explicit PATH argument that does not exist).

# Detection design

## Step 1 — candidate identification

A **bare-variable assignment** `<ident> = AsyncMock()` or `<ident> = MagicMock()` on its own line is a candidate — the process-standin object itself, **not** a nested `<ident>.stdout = MagicMock()` sub-stream assignment (a pipe object has no `.pid` semantics of its own, so flagging those would be a category error, not merely a false positive).

## Step 2 — the forward window (20 lines)

Within the 20 lines immediately after the candidate assignment, the gate requires **all** of, scoped to the SAME identifier:

1. an explicit `<ident>.returncode = None` assignment (**required precondition** — see "Why `returncode = None` is required" below);
2. at least one process-lifecycle/streaming indicator: `<ident>.stdout.readline`, `<ident>.stderr.read`, `<ident>.wait`, `<ident>.kill`;
3. **absence** of `<ident>.pid = <int-literal>` (a positive or negative integer literal — `mock.pid = 12345` is the sanctioned hardening).

## Step 3 — the killpg-patch exemption (±30 lines)

A hit that fails step 2 is still **skipped** (not flagged) when a real (non-comment) `patch(...)` / `patch.object(...)` call naming the string literal `"killpg"` (or `'killpg'`) appears anywhere within 30 lines before OR after the mock-assignment line.

## Why `returncode = None` is required (evidence, not a guess — §11.4.6)

Both call sites in `search.py` that can EVER reach `os.killpg` / `os.getpgid` are lexically gated behind the identical precondition:

```
if proc.returncode is None:
    ...
    _pid = proc.pid
    if isinstance(_pid, int) and _pid > 1:
        ...
        os.killpg(_pgid, _signal.SIGKILL)
```

`proc.returncode is None` is the `asyncio.subprocess` convention for "process still running" — it is the **only** precondition under which the cleanup/kill branch is reachable at all, independent of the pid guard. A mock whose `.returncode` is left unset (an auto-generated `MagicMock` instance is not `is None`) or is explicitly set to a real exit code (`0`, a positive int, or a variable that defaults to one) can therefore **never** reach the kill branch — flagging such a mock for lacking `.pid = <int>` would be a false-positive refusal on code that is structurally incapable of the BOB-126 defect (a §11.4.201 FAIL-bluff, forbidden exactly as a PASS-bluff is).

This was proven by reading `search.py` directly and cross-checked against **every** existing `AsyncMock()`/`MagicMock()` subprocess-standin in `tests/**/*.py` as of this gate's authoring date. Three pre-existing, genuinely safe helper factories set `.returncode` to a completed-process value and have no reachable kill path — they must not be flagged, and are not, under this rule:

| File | Helper | .r sh |
|---|---|---|
| `tests/unit/merge_service/test_deadline_tunable.py:41` | `_proc_mock()` | = re (p de |
| `tests/unit/merge_service/test_edge_case_challenges.py:137` | `TestPublicTrackerResultParsing._proc_mock()` | = re (p de |
| `tests/unit/test_merge_trackers.py:~247` | `_streaming_proc_mock()` | = (h |

**Verified via paired mutation** (§1.1): temporarily disabling the `returncode = None` requirement causes the gate to WRONGLY flag exactly these three known-safe sites on the real tree — proving the check is load-bearing, not decorative.

## Why `killpg`-alone (not `killpg` AND `getpgid`) is a sufficient exemption

`os.killpg` is the **only** destructive operation in the `search.py` cleanup path. `os.getpgid` is a read-only lookup that is itself gated behind the identical `isinstance(_pid, int) and _pid > 1` guard before it can ever run with a test-supplied pid (verified by reading `search.py` directly) — so a real `os.killpg` patch alone makes the actual syscall unreachable regardless of what `.pid` coerces to. This resolves a real, evidence-confirmed case in the current tree: `tests/unit/merge_service/test_deadline_tunable.py::test_bob126_regression_deadline_path_never_calls_killpg_with_pgid_le_1` patches `os.killpg` alone (deliberately, per its own docstring — "these MUST BE PATCHED here — this test intentionally exercises the vulnerable code path"), never `os.getpgid`, and is the RED-first regression guard for the production-side pid guard itself.

A patch mentioned only inside a **comment** (documenting a nearby test's patches, e.g. prose reading "patched getpgid") is explicitly **not** counted as a real patch — matching a mention of the pattern as if it were the pattern is exactly the carrier footgun §11.4.196(D)/§12.12/§11.4.201(6)-(7) name; every candidate patch line is re-checked to exclude comment-only lines before it counts.

### Honest boundary (§11.4.6)

A mock whose `.returncode` is assigned **indirectly** — e.g. `mock.returncode = returncode` where a FUTURE caller passes `returncode=None` — is invisible to this purely lexical check. This gate proves the **literal shape currently on disk** is safe, not every possible call-site permutation across function boundaries. The sibling production-side gate (`check_cm_killpg_pgid_guard.sh`) remains the primary, always-active defense regardless of what any test constructs.

## Self-exclusion

Unlike the sibling production gate, this gate scopes to `tests/**/*.py` and is itself a `.sh` file living outside `tests/` — it never scans itself, and its own paired meta-test (also a `.sh` file) is likewise outside the `*.py` scope. No structural self-exclusion is needed.

## Side-effects

None. The gate is read-only: it never modifies any scanned file. It creates one `mktemp` findings file for its own run and removes it on exit via `trap ... EXIT`.

## Dependencies

- `bash` (uses `[[ ]]` with a regex-variable `=~` match, `<<<` here-strings, process substitution).
- `grep -E`, `sed -E`, `sed -n`, `find`, `wc -l` — all standard on the project's supported hosts.
- No Python, no network access, no repository write access.

# Cross-references

- **BOB-126** — the forced-logout incident chain (BOB-116/120/123/124/125/126) this gate's real-tree pattern was extracted from.
- **`scripts/pre_build/check_cm_killpg_pgid_guard.sh`** (CM-KILLPG-PGID-GUARD) — the sibling production-side gate; together the two gates cover both halves of the codebase (production excluded from `check_cm_killpg_pgid_ guard.sh`'s scope, tests excluded from production's).
- **§11.4.238** — QA-discovery-channel mandate: this gate is the "new or strengthened automated check" a coverage-escape audit requires for a defect first found out-of-band.
- **§11.4.226** — evidence-class-at-closure: a standing guard, not merely a source-side comment, is what keeps a fix from silently reopening.
- **§11.4.240** / **§11.4.249** — producer≠verifier / four-role separation: the test-authoring agent and this gate are structurally separate checks; a regression that quietly reintroduces an unguarded subprocess mock is caught by an INDEPENDENT mechanism, not by the same hand that wrote the test.
- **§11.4.115** / **§11.4.135** — RED-first meta-test discipline / permanent regression-guard suite: `tests/pre_build/test_check_cm_test_mock_pid_explicit_int.sh` is this gate's paired §1.1 mutation harness.
- **§11.4.196(D)** / **§12.12** / **§11.4.201(6)-(7)** — the "instrument matches a MENTION of its own carrier text (a comment), not the real thing" footgun class the killpg-exemption's comment-line exclusion closes.
- **§11.4.201** — every guard/gate asserts the real condition; a false-positive refusal is a FAIL-bluff exactly as forbidden as a false-negative pass (the rationale for the `returncode = None` precondition and the killpg-alone exemption).
- `download-proxy/src/merge_service/search.py:1190-1240` — the real production cleanup code whose exact control-flow shape (`if proc. returncode is None:` gating both the pid-guard and the kill call) this gate's detection design is derived from.
- `tests/unit/merge_service/test_deadline_tunable.py::test_deadline_hit_flag_true_when_readline_times_out` — the fixed test this gate's real-tree acceptance criterion cites (both `mock.pid = 12345` AND `patch.object(_search.os, "killpg")` present).

## Companion files

- `scripts/pre_build/check_cm_test_mock_pid_explicit_int.sh` — the gate itself.
- `tests/pre_build/test_check_cm_test_mock_pid_explicit_int.sh` — the §1.1 paired meta-test: `golden-good-1` (explicit `mock.pid = 12345`), `golden-good-2` (no pid, but `os.killpg` AND `os.getpgid` both real-patched), `golden-bad` (the raw pre-fix BOB-126 shape — no pid, no killpg patch), a real-tree smoke run against `tests/**/*.py`, and a no-op-stub negative control (CONST-XII anti-bluff — a gate that always exits 0 is caught wrongly-passing the golden-bad fixture).

## Last verified

2026-08-19 — `bash tests/pre_build/test_check_cm_test_mock_pid_explicit_int.sh` ran GREEN (5/5: golden-good-1 rc=0, golden-good-2 rc=0, golden-bad rc=1, real-tree default scope rc=0, no-op-stub negative control confirmed); `shellcheck` clean on both scripts with zero findings. Paired-mutation sanity checks confirmed both the pid-guard recognition AND the `returncode = None` precondition are load-bearing: disabling the pid-guard match flips `golden-good-1` to a wrong FAIL; disabling the `returncode = None` precondition flips the real-tree run to a wrong FAIL on exactly the three known-safe helper factories documented above.